

Cadence NC 仿真工具使用教程

曾键旭2016-10

Cadence NC 仿真工具使用教程

- 一、文件准备.....1
 - 1.建立工作目录.....1
 - 2.准备文件.....2
- 二、使用流程简介.....2
 - 1.工作模式选择.....2
 - 2.建立工作环境.....3
 - 3.Compile.....5
 - 5.Simulate6
- 三、门级仿真.....6
 - 1.文件准备.....6
 - 2.编译标准单位库.....6
 - 3.编译门级网单和 testbench.....7
- 四、SDF 反标8
 - 1.什么是 SDF 文件？9
 - 2.怎样得到 SDF 文件？9
 - 3.Verilog Testbench 的 SDF 反标.....9
 - 4.VHDL Testbench 的 SDF 反标9
 - 5.在 Modelsim 中进行 SDF 反标10
- 五、注意事项.....10
- 附录：实验用代码.....19

一、文件准备

1.建立工作目录

在LINUX系统下打开一个控制台窗口，建立用来进行实验的项目目录（例如Digital_IC），命令如下。

```
cd
mkdir test
cd test
mkdir work
pwd
ls
```

```
进入用户的根目录
建立 test 目录
进入 test 项目目录
建立用于保存各种临时文件的 work 目录
显示当前目录，确认没有错误
显示当前目录下的所有文件
```

2.准备文件

本实验使用的设计是一个4位计数器，在/home/ic/data/目录下已经有参考源代码，可以把它直接拷贝到当前工作目录，命令如下：

```
pwd                                确认当前目录是test项目目录
cp/home/ic/data/demo-updown-cnt.v  把参考源代码拷贝到当前目录（即最后的“.”）
cp/home/ic/data/demo-updown-cnt-tb.v 把参考源代码拷贝到当前目录（即最后的“.”）
ls                                  再次确认拷贝了文件
copy报错的话可以打开/home/ic/data复制文件
```

二、使用流程简介

1.工作模式选择

打开terminal，输入nclaunch &打开软件。第一次运行时，将出现图 1 的对话框，需要用户选择工作模式，这里可供选择的主要是 Multiple Step 和 Single Step，请选择 Multiple。

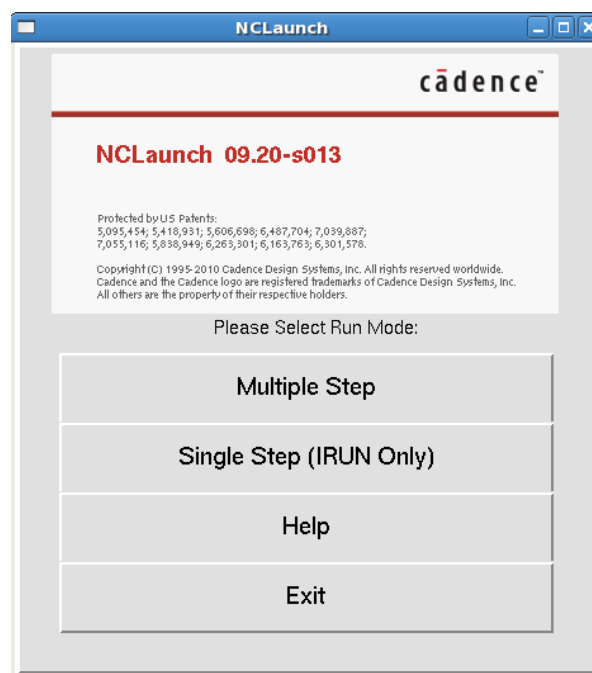


图1 运行模式对话框

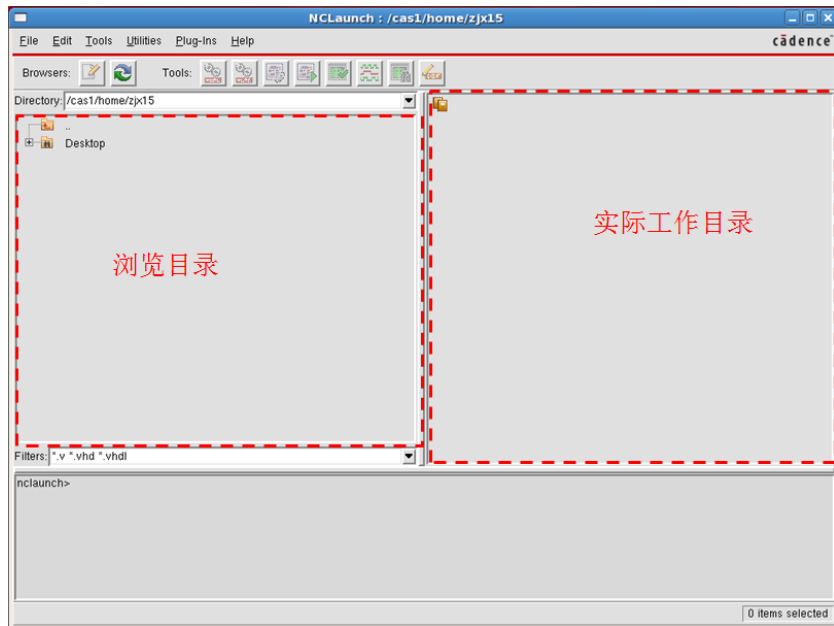


图2

2.建立工作环境

点击File set design directory

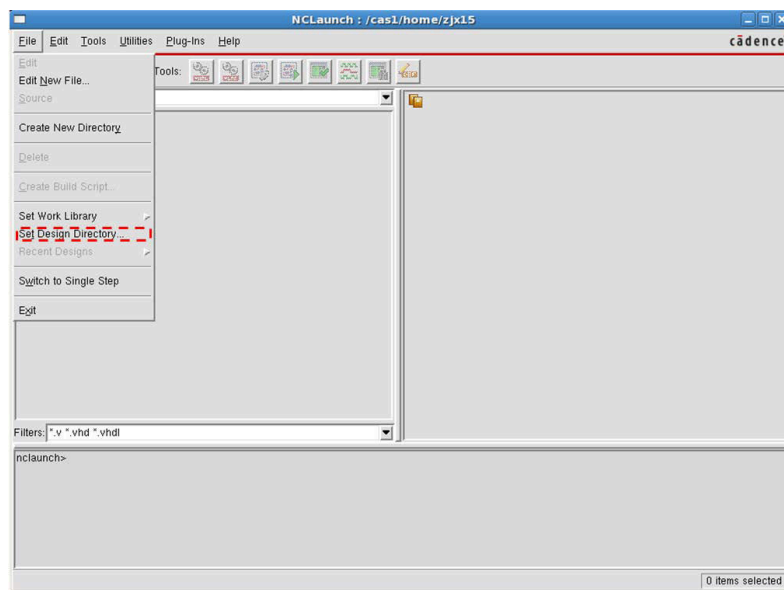


图3

新建文件夹，选择刚刚创建的test，然后点击 Create cds.lib File, Save, 出现图5弹窗，选择默认第一个选项即可，然后点击OK。

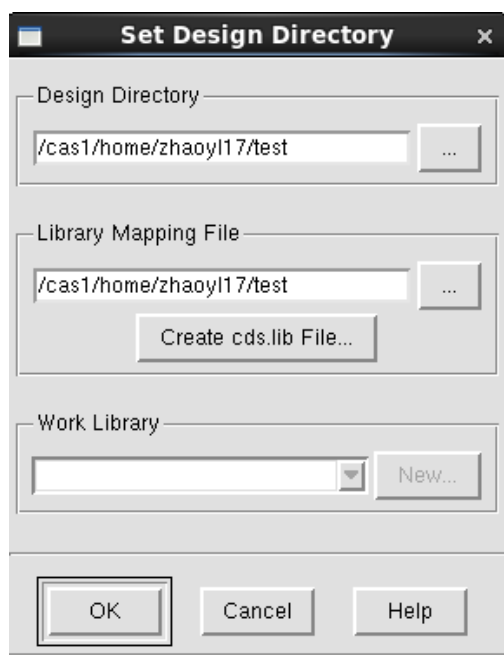


图4

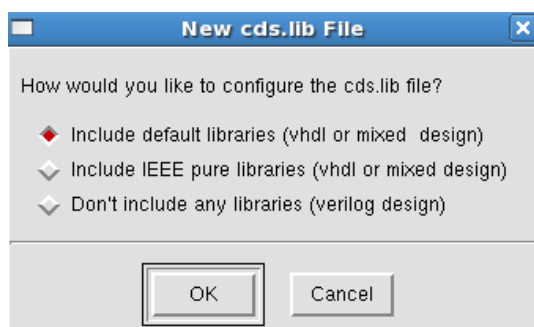


图5

创建结束后会发现之前的两个verilog源代码已经出现在左边窗口，如图6所示：

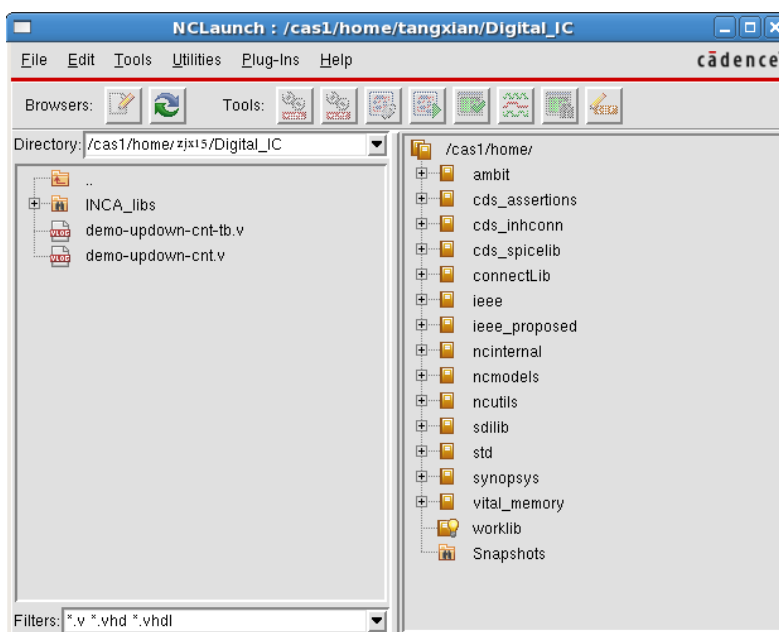


图6

3.Compile

先编译源文件，再编译tb文件。编译方法：选中文件，点击上一排的编译按钮，。

如果出现语法错误，控制窗口中会以红色内容进行提示，修改完毕后才可以进行下一步。

编译后生成的文件如图8，updown_cnt对应源文件，tb对应testbench。

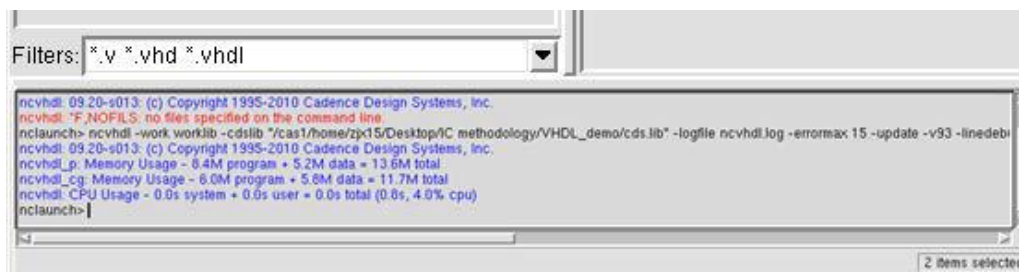


图7

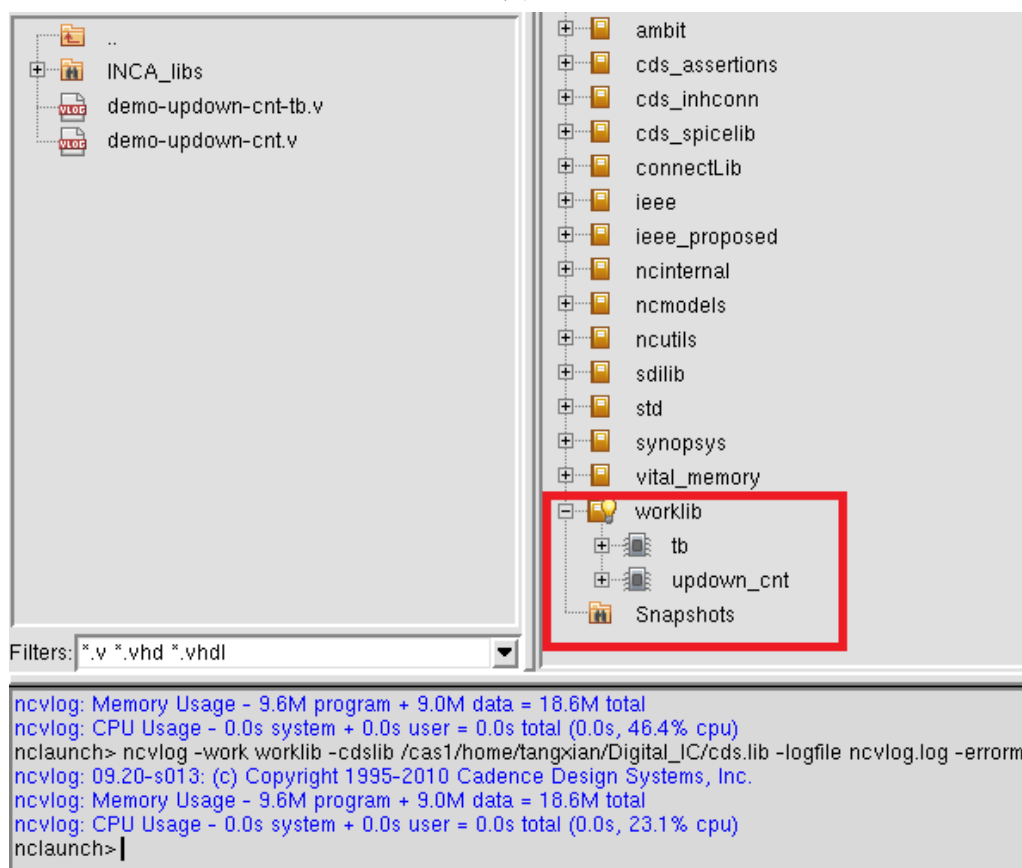
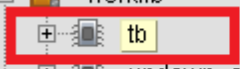
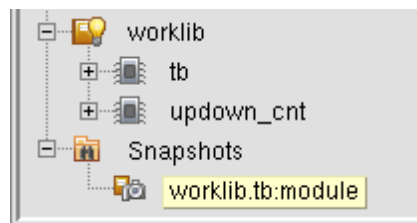


图8

4.Elaborate

选中生成的tb文件，点击，生

成snapshot下的文件



5.Simulate



选中worklib.tb:module，点击进入simulator。

Simulator中波形的观察，请参考本文第二部分内容。

三、门级仿真

1.文件准备

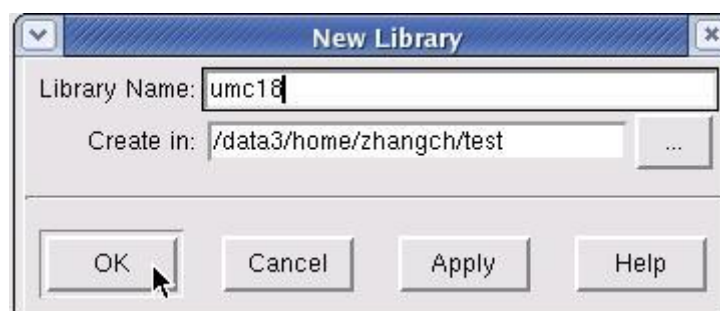
对综合后输出的网单文件（例如 demo-counter-post.v）进行门级仿真，使用在功能仿真中使用的testbench（例如demo-tb.vhd）。

另外，还需要把标准单位的仿真库文件（/home/ic/library/umc18.v）准备好。（可以拷贝到当前项目目录里）

```
zhangch@jupiter:~/test
[zhangch@jupiter test]$ ls de*
demo-counter-post.sdf  demo-counter.vhd
demo-counter-post.v   demo-tb.vhd
[zhangch@jupiter test]$
[zhangch@jupiter test]$ ls *.v
demo-counter-post.v  umc18.v
[zhangch@jupiter test]$
```

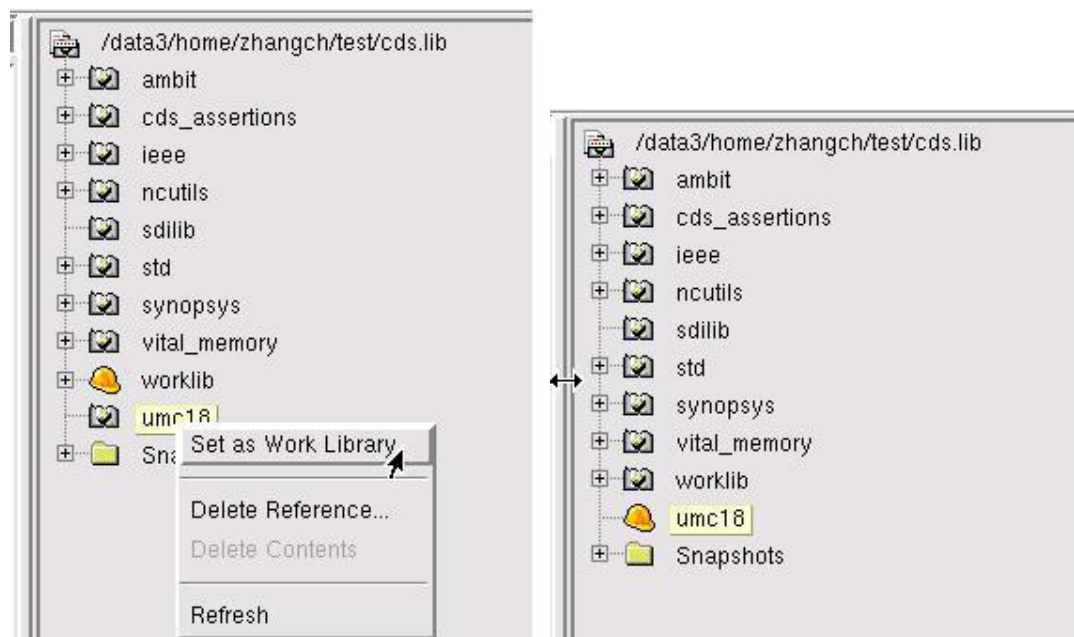
2.编译标准单位库

1) 选择“Edit -> Add -> New Library”菜单，建立用于保存编译后的库文件的库。在“Library Name”处输入“umc18”（可以随便起一个名字）

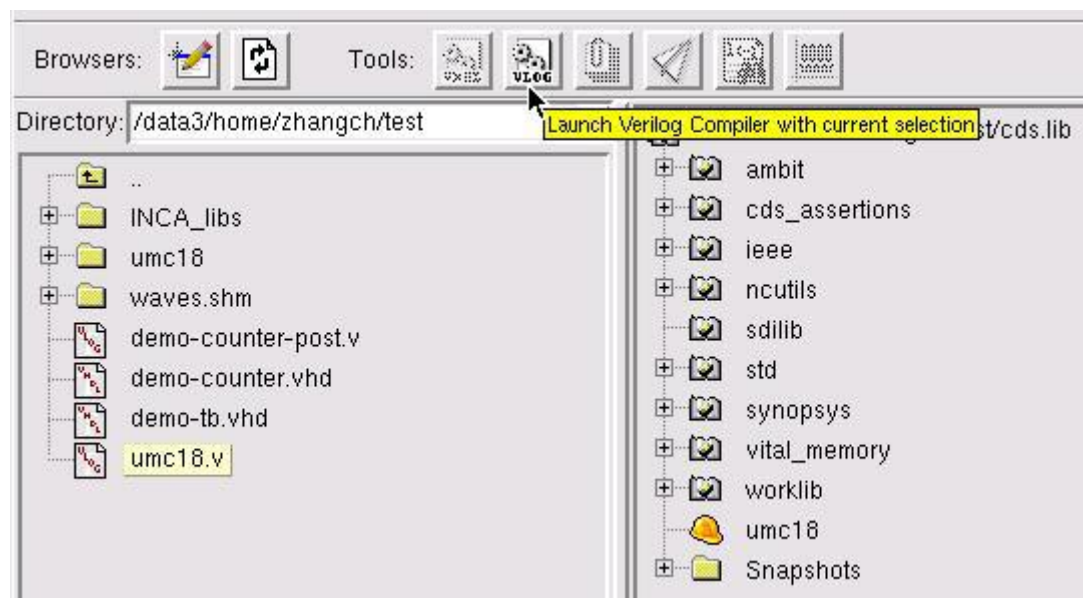


2) 选择“File -> Set Work Library”菜单把刚才建的库（umc18）设为工作库（也可以

在umc18 库上按鼠标右键）。设置好以后“umc18”会变成菊黄色。

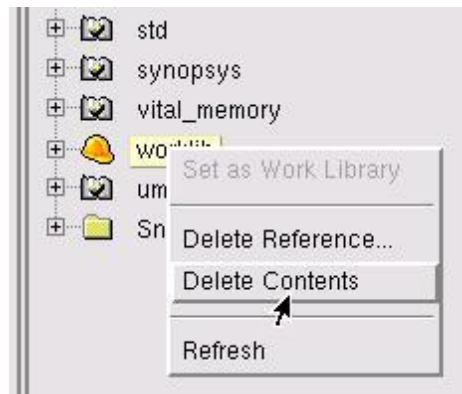


3) 选中umc18.v 后，选择“Tool -> Verilog Compiler”菜单编译“umc18.v”



3.编译门级网单和 testbench

- 1) 选择“File -> Set Work Library->worklib”菜单，重新把worklib 设为工作库
- 2) 选择“Edit -> Delete -> Library Contents”菜单，清除功能仿真时编译后的文件。（也可以在库管理器的worklib 上直接按鼠标右键，选择“Delete Contents”）

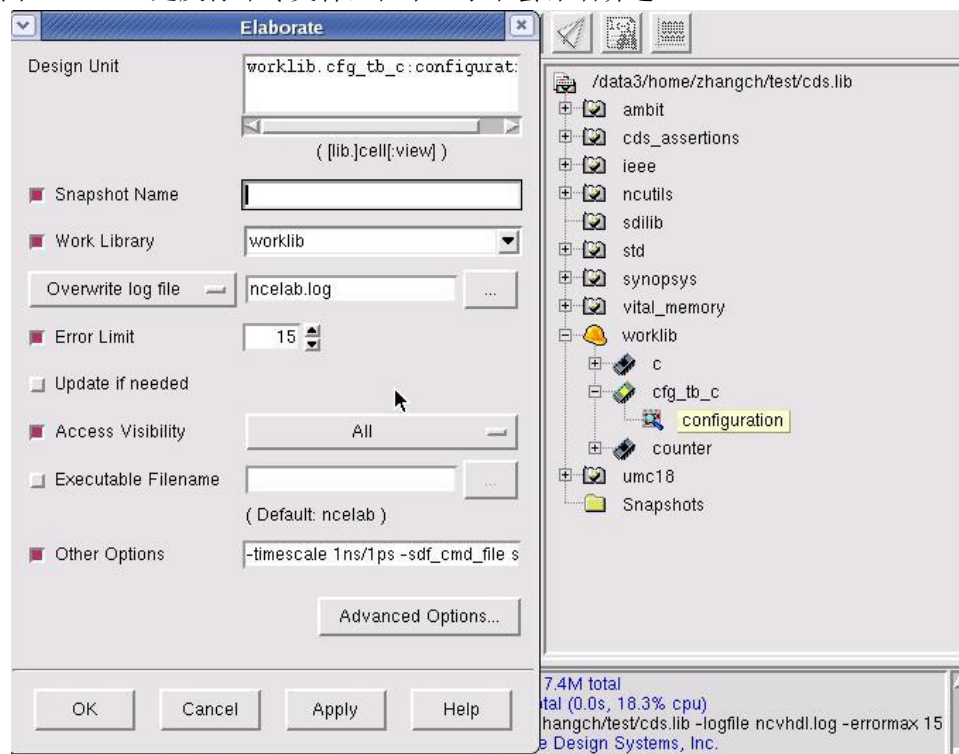


3) 编译门级网单 (demo-counter-post.v) 和 testbench (demo-tb.vhd)

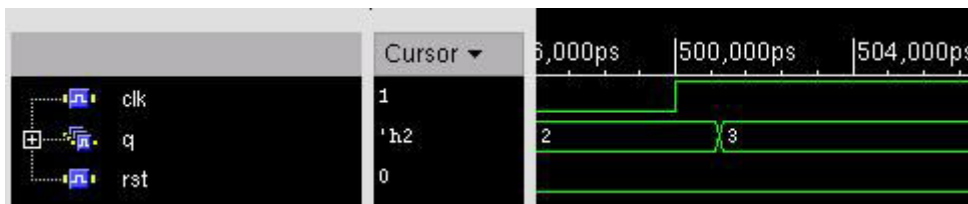
4) Elaborate 时打开选项“Other Options”，输入“-timescale 1ns/1ps”。

如果想进行 SDF 反标，输入“-timescale 1ns/1ps -sdf_cmd_file sdf.cmd”；

其中“sdf.cmd”是反标命令文件，在下一小节会详细讲述。



5) 下面的仿真过程和功能仿真一致，门级仿真的输出波形在没有进行SDF 反标时可以观察到约1ns 的门延时。



四、SDF 反标

为了得到精确的门级仿真或后仿真，需要对网单进行 SDF 标注。

1.什么是 SDF 文件？

SDF全称是Standard Delay Format，它是一种用于表示电路系统延时信息的文本格式，在电路设计的后仿真中是不可或缺的。SDF 是IEEE 的标准（标准号IEEE 1497-2001），各种验证工具都可以使用。

2.怎样得到 SDF 文件？

在Design Compiler综合以后用write_sdf 命令导出。例：write_sdf xxx.sdf

3.Verilog Testbench 的 SDF 反标

如果 testbench 是用verilog 写的，则在其中initial 之后加入下列语句即可：

\$sdf_annotate ("xxx.sdf", top_module_name);

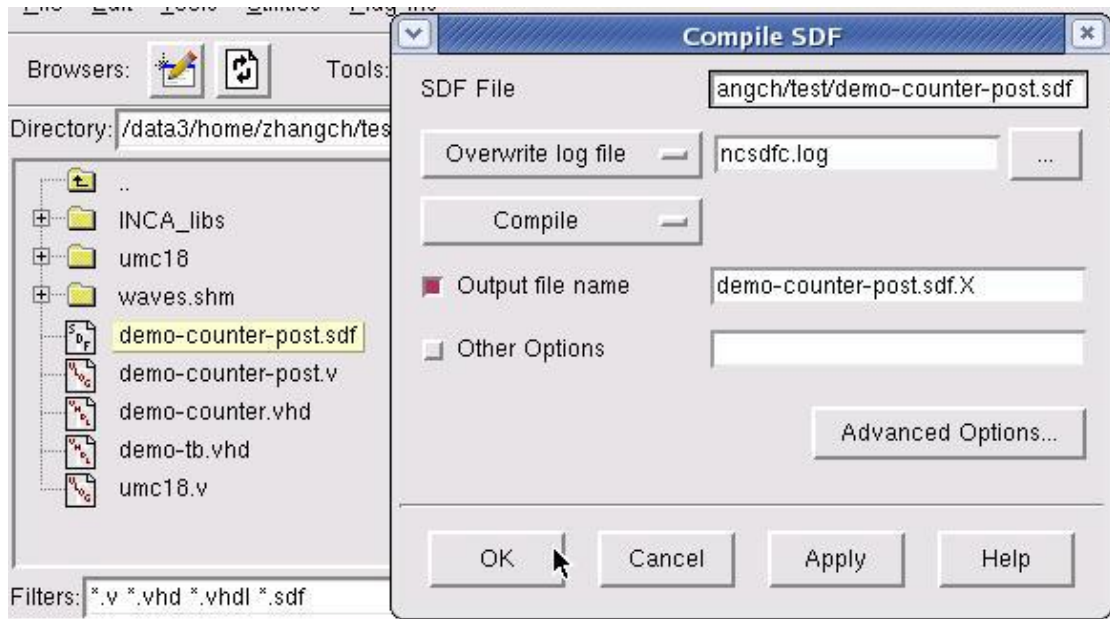
xxx.sdf 表示SDF 文件名称，top_module_name 表示sdf 所要标注的模块名称，如果有多层结构，中间用“.”隔开，如top_module_name.sub_module_name。

4.VHDL Testbench 的 SDF 反标

在 NC 中做VHDL Testbench 的SDF 反标需要三步：

1) 编译SDF 文件。

选择菜单Tools > SDF Compiler ...，在弹出的窗口中填好SDF File（例如demo-counter.post.sdf）以及Output file name。



点击 OK 即可编译得到demo-counter-post.sdf.X 文件。

2) 写一个SDF 命令文件（例如起名为sdf.cmd），在其中标明编译后的SDF 文件名以及所要反标的单元名称，一个简单的SDF 命令文件如下：

```
COMPILED_SDF_FILE = "demo-counter-post.sdf.X",
```

```
SCOPE = :uut_a;
```

第一项是编译后的SDF 文件名，第二项是所要反标的单元名称。“:u0”是该单元在testbench 中的名字。一般在testbench 中如下的映射语句：

uut_a: counter port map (clk,rst,q);

SDF 最后就反标到u0 单元上。如果有多层结构，中间用“:”隔开。还要注意SDF 命令文件中最后一句句末用分号，其他句末用逗号。把该SDF 命令文件保存为sdf.cmd。

3) Elaborate 时打开选项“Other Options”，输入：

“-timescale 1ns/1ps -sdf_cmd_file sdf.cmd”；

注意：因为Design Compiler 导出的SDF 文件不是很地道，有时会出现类似下面的警告：
ncelab: *W,SDFNEP: Failed Attempt to annotate to non-existent path (IOPATH (posedge A)
Y) of instance :U0.U123 of module XOR2X1 <./data/rctrl_new.sdf, line 133>.
可忽略。

5.在 Modelsim 中进行 SDF 反标

如果用 modelsim 仿真，则点击菜单Simulate > Start Simulation ...，在弹出窗口中有SDF 的标签，在该标签点击Add ...，设置好SDF 文件名及所要标注的模块名即可。

五、注意事项

1.关于 timescale

有些机器会提示timescale的错误，解决方案如下：

NC 缺省情况下要求所有的 VERILOG 文件都有“timescale...”这一行，否则在elaborate 时会报错。为避免 elaborate 时出现关于 timescale 的错误，在elaborate时打开选项“enable the Other Options”，输入“-timescale 1ns/1ps”，表示缺省的时间单位1ns，时间精度1ps。

2.关于关闭 timing check

在仿真时可以关闭“timing check”，例如 setup/hold time 的检查，以提高仿真的速度在 elaborate 时打开“-notimingchecks 顶层模块名”选项，或用下述命令行方式运行：
% ncelab -notimingchecks top

3.关于 NC 仿真多次以后打开窗口变慢，有时要等半分钟以上才能打开窗口的问题

有些同学使用 gedit 编辑源代码，NC 和 gedit 文本编辑工具有冲突。所以出现以上问题时把 gedit 窗口暂时关闭，重新进行仿真即可。

SimVision-tutorial

王亚 2014-10-9

SimVision-tutorial

- 1、SimVision 窗口12
- 2、观察信号波形.....12
- 3、仿真与仿真时间设置.....13
- 4、波形窗口13
- 5、原理图窗口16
- 6、源代码窗口16
- 7、寄存器窗口17
- 总结.....19
- 附录：实验用代码.....19

1、SimVision 窗口

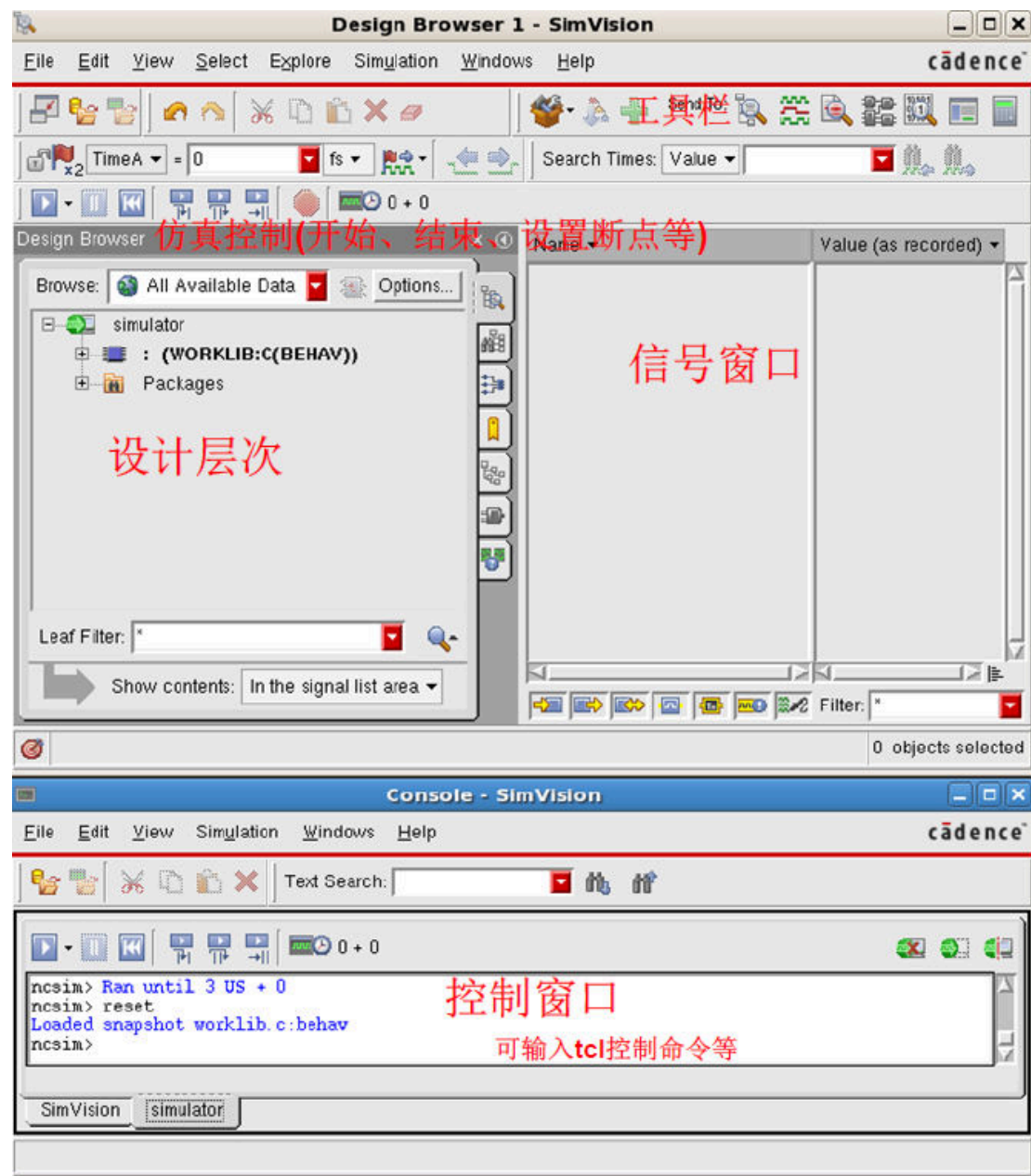


图1

SimVision 窗口如图1 所示，以下用一个来说明SimVision 中窗口的使用

2、观察信号波形

在Design Brower-SimVision 窗口左侧显示设计层次窗口中选择实体，该模块中信号会显示在右侧信号窗口中，选中要观察的信号(Shift 连选, Ctrl 多选)，右键选择Send to waveform window，如图2 所示，仿真器会弹出波形窗口（图3）

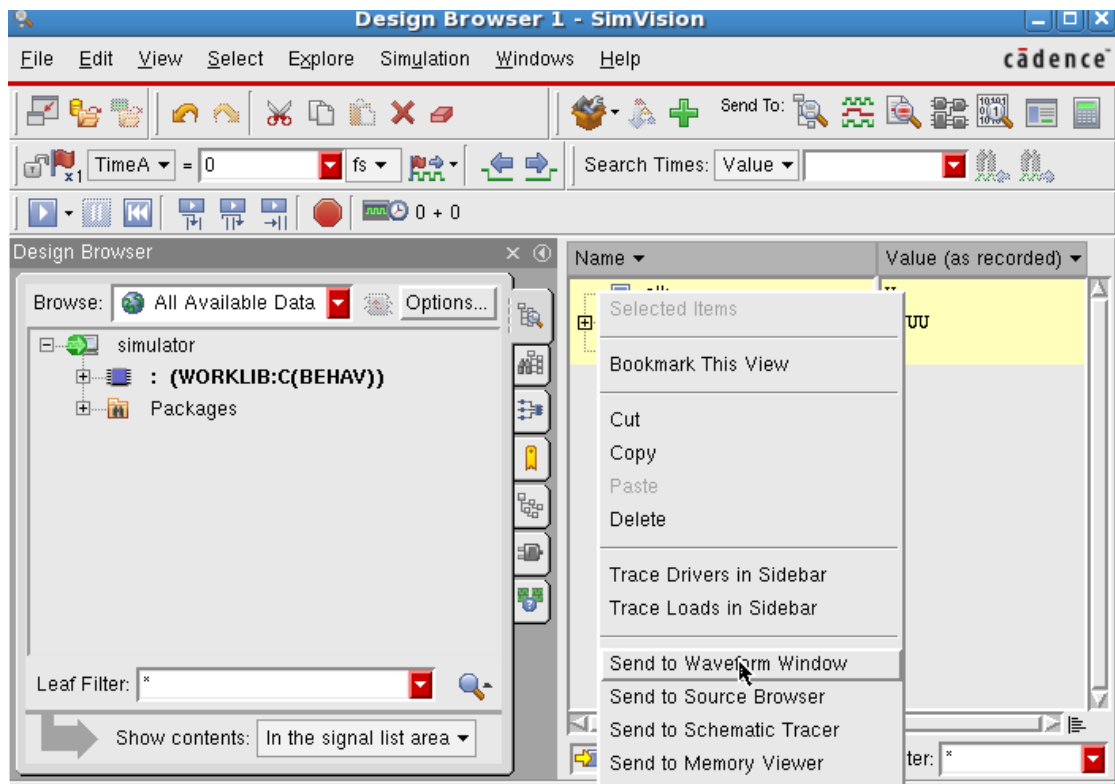


图2

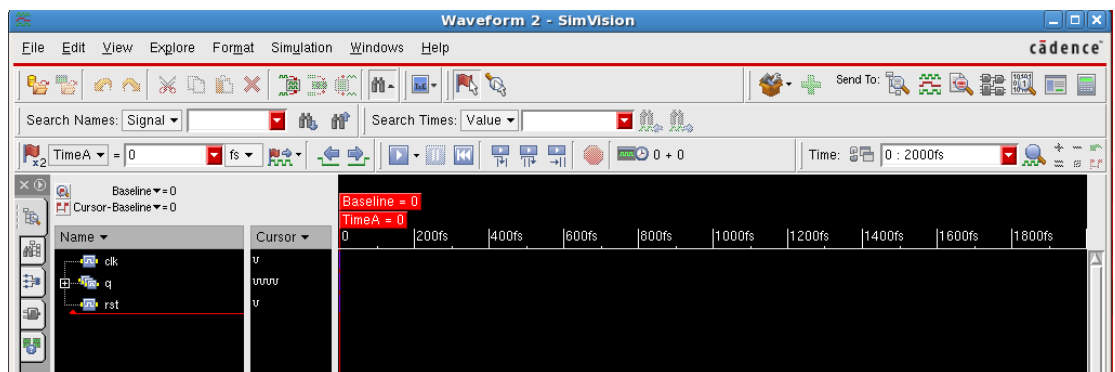


图3

3、仿真与仿真时间设置

(1)选择Simulation-Run 开始仿真，Simulation-Stop 结束仿真；

(2)点击  开始仿真，点击  结束仿真；

(3)在控制窗口Console-simulator 输入命令也可令仿真开始或停止，输入run+时间，可以控制仿真的时间，如run 1 ms。

4、波形窗口

仿真一段时间后，可以看到波形窗口出现波形，如图4。

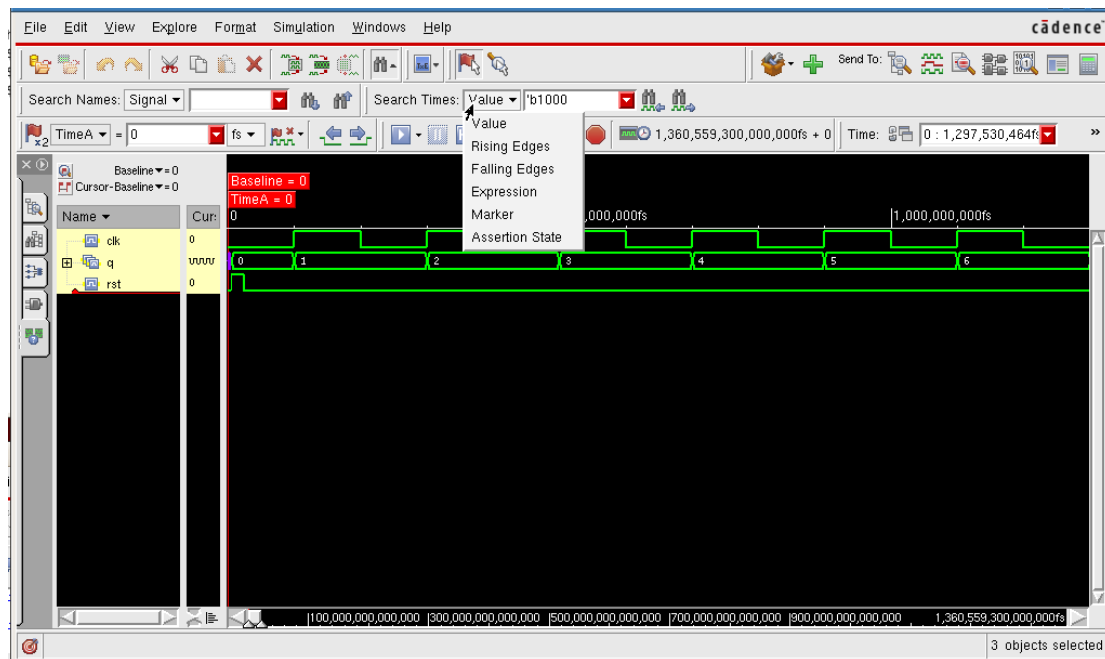


图4

- (1) 按住Ctrl键，上下滑动鼠标滚轮，可以对波形图进行放大或缩小；
- (2) 在图4中红色圆框中输入信号名(如q)，可以查找到相应信号，适用于波形图中波形文件较多时快速定位至所需要观察的信号；
- (3) 在Search Times选项后点下拉菜单，可以查找信号值等于某一个值，或信号的上升沿下降沿时间等，也可输入公式查找；
- (4) 在TimeA=后框中输入时间，或在图形窗口中直接拖动红色旗帜状TimeA，可以改变TimeA的值，选择TimeA下拉菜单New cursor，可以设置新的TimeB，如图5，赋值方式同上；

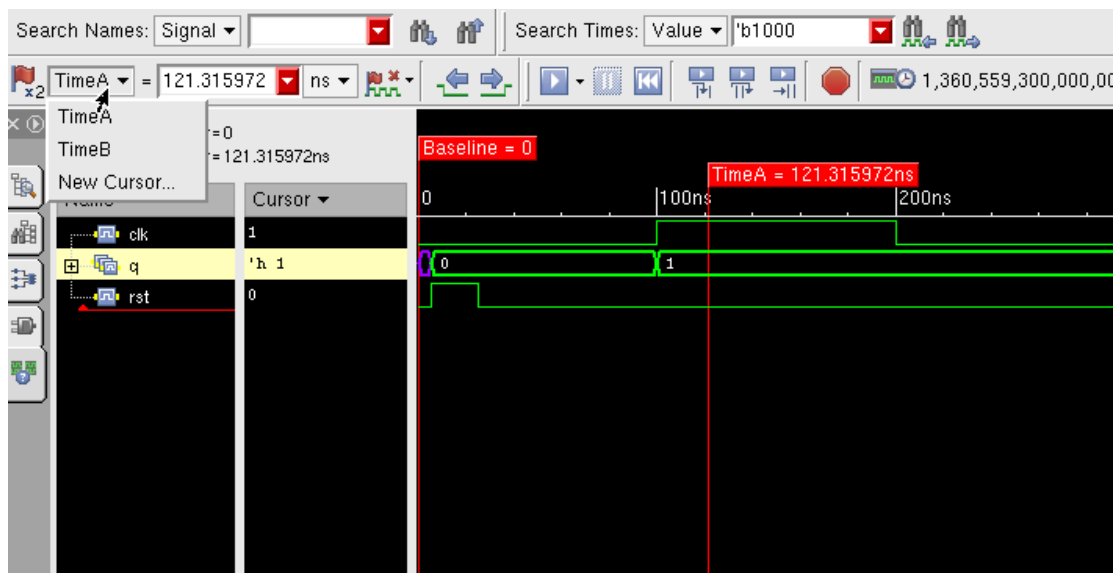


图5

- (5) 指定范围信号放大，如图 6 所示，可任选开始时间(当前时间轴TimeA/B，取决于上方框中，Baseline)与结束时间，选择后，选OK，点选图示中按钮完成放大；

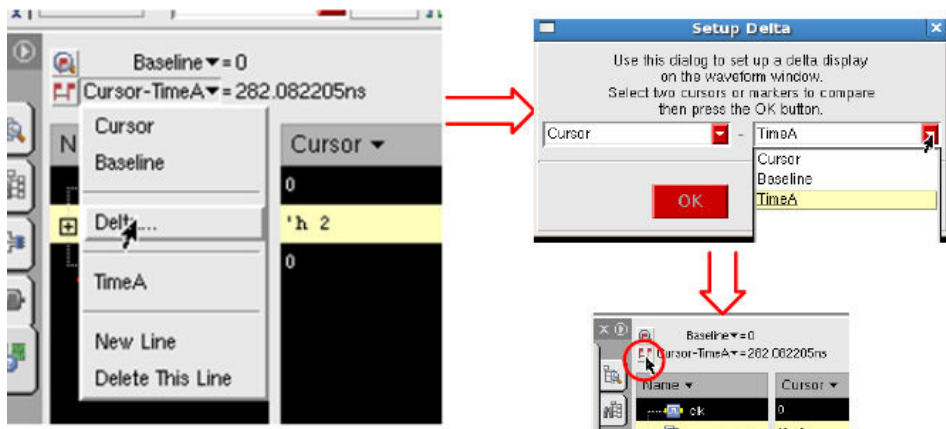


图6

(6) 修改信号显示格式

图形窗口中可改变信号显示格式(二进制、十六进制等), 具体操作为选中一个信号, 在窗口第二列单击右键, 在弹出菜单中选择Radix/Mnemonic 项, 选择自己需要的显示格式, 如图7 所示;

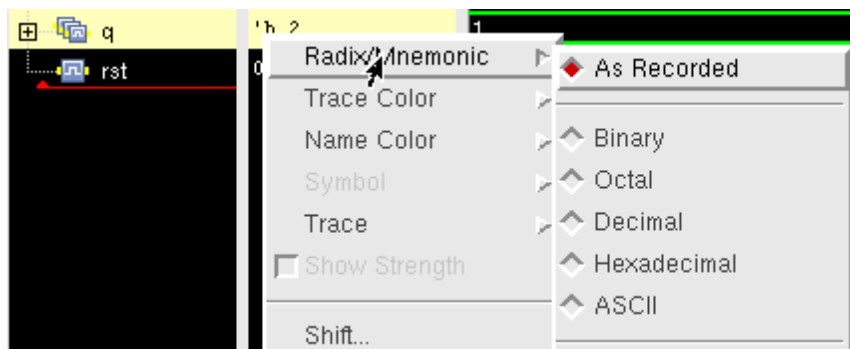


图7

(7) 改变信号顺序, 用鼠标中键选中信号, 拖动至需要位置放开即可;

(8) 创建 bus。在调试过程中, 有时我们需要观测某几位信号组成数组后的值, 这就要求我们可以创建bus, 可用鼠标左键选中若干个信号, 如q[3]、q[1], 单击鼠标右键, 选择Creat Bus 即可, 如图8 所示, 如不再需要, 可右键删除或ungroup。

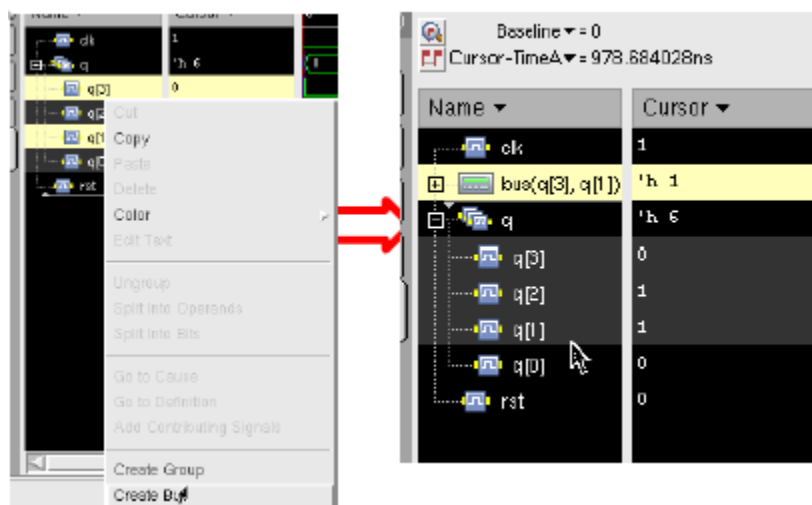


图8

5、原理图窗口



选中uut，单击SimVision 工具栏中的，可打开原理图(schematic tracer)窗口，双击原理图窗口的模块可进入模块内部，如图9 所示。

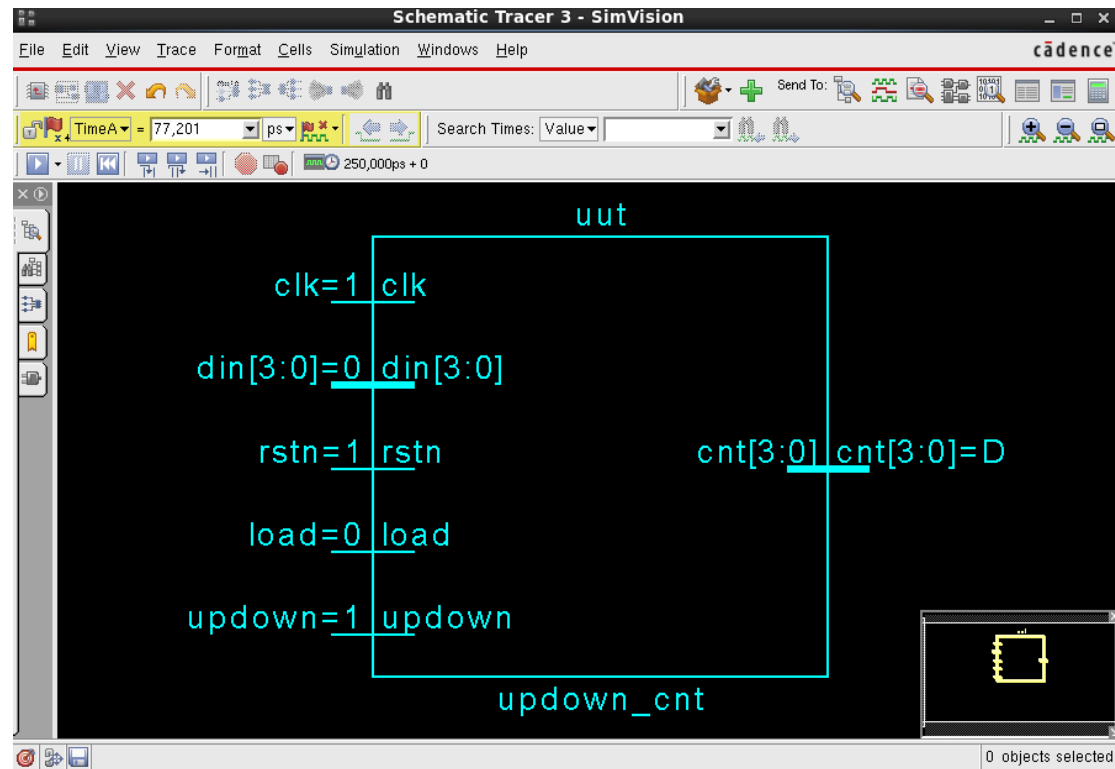


图9

图右下方可以实现对原理图各位置的快速定位、放大与缩小等功能，需要特别指出的是，SimVision 中各工具是内部链接的，在其中一个工具中选中某一信号后，其他工具也会相应选中，如，这里在原理图窗口选中信号Clk，则在波形图窗口中Clk 同样被选中，SimVision 的这个功能可以实现交互式图形的快速查找，非常方便。

6、源代码窗口



选中uut，在SimVision 工具栏中单击，可打开源代码窗口(source browser)，如图10 所示。图中下拉菜单部分可以快速定位代码module，在多层次设计中可以比较方便的找到要找的模块。源代码窗口也与其他窗口内部链接，可实现信号选择等功能。如果实际调试过程中发现代码bug，可以直接在源代码窗口中点击编辑源代码，编辑完成保存后点击Simulation-Reinvoke Simulator 重新编译和描述设计并重新启动仿真器(等待时间可能有点久)，原来调试过程中做的保存信号与创建bus等工作在重启后全部自动打开，这时只需要重新跑一次仿真就可以验证修改后的代码是否有错。

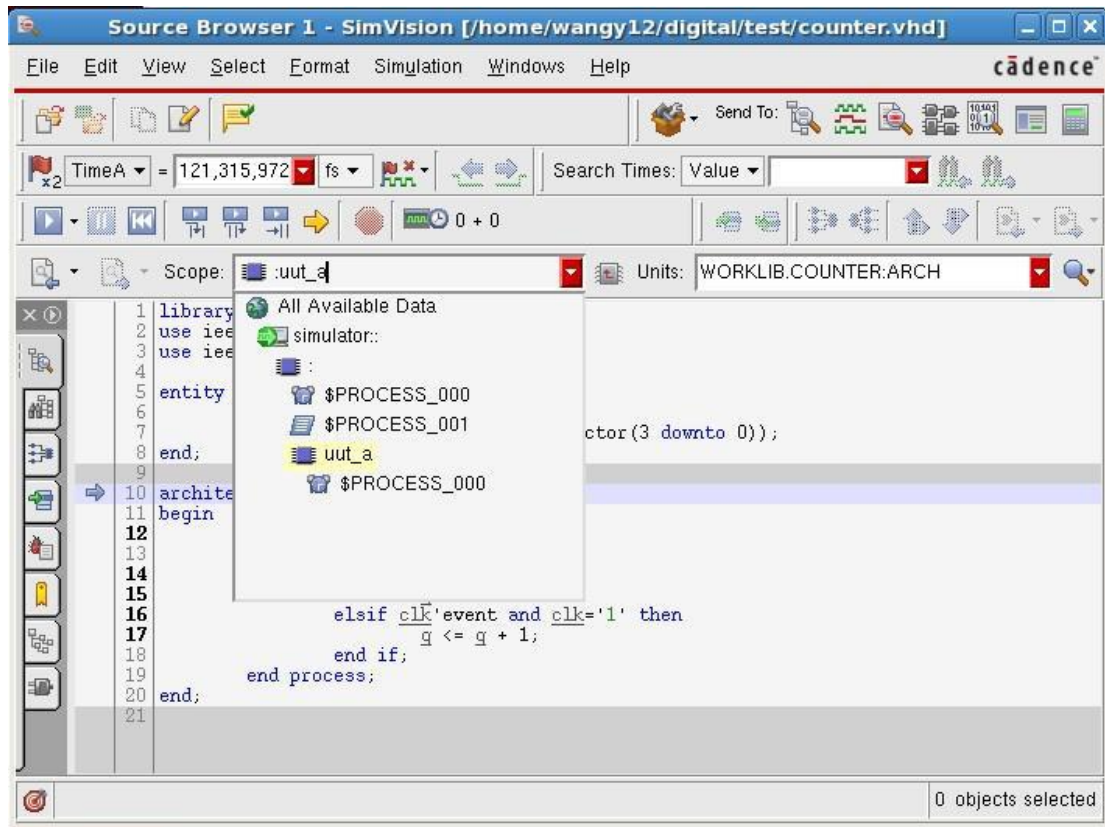


图10

7、寄存器窗口

(1)在waveform window 选中需要观测的信号，点击工具栏中send to target register window 选项，如图11 所示，弹出寄存器窗口如图12 所示。

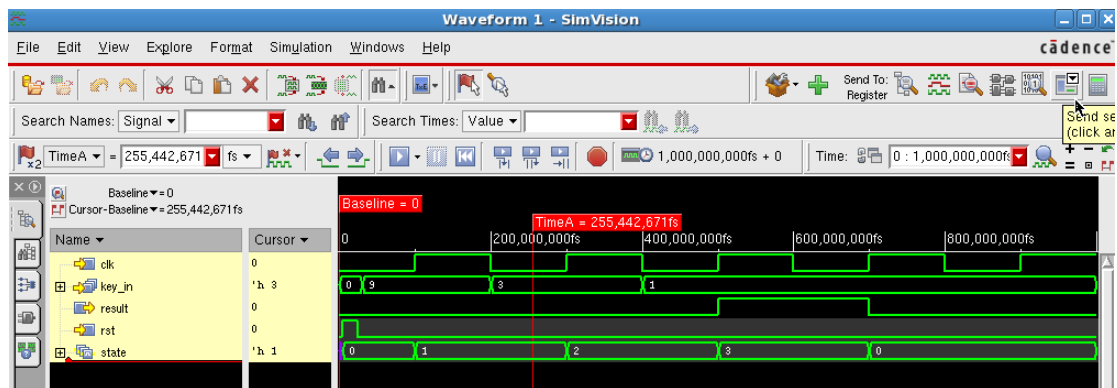


图11

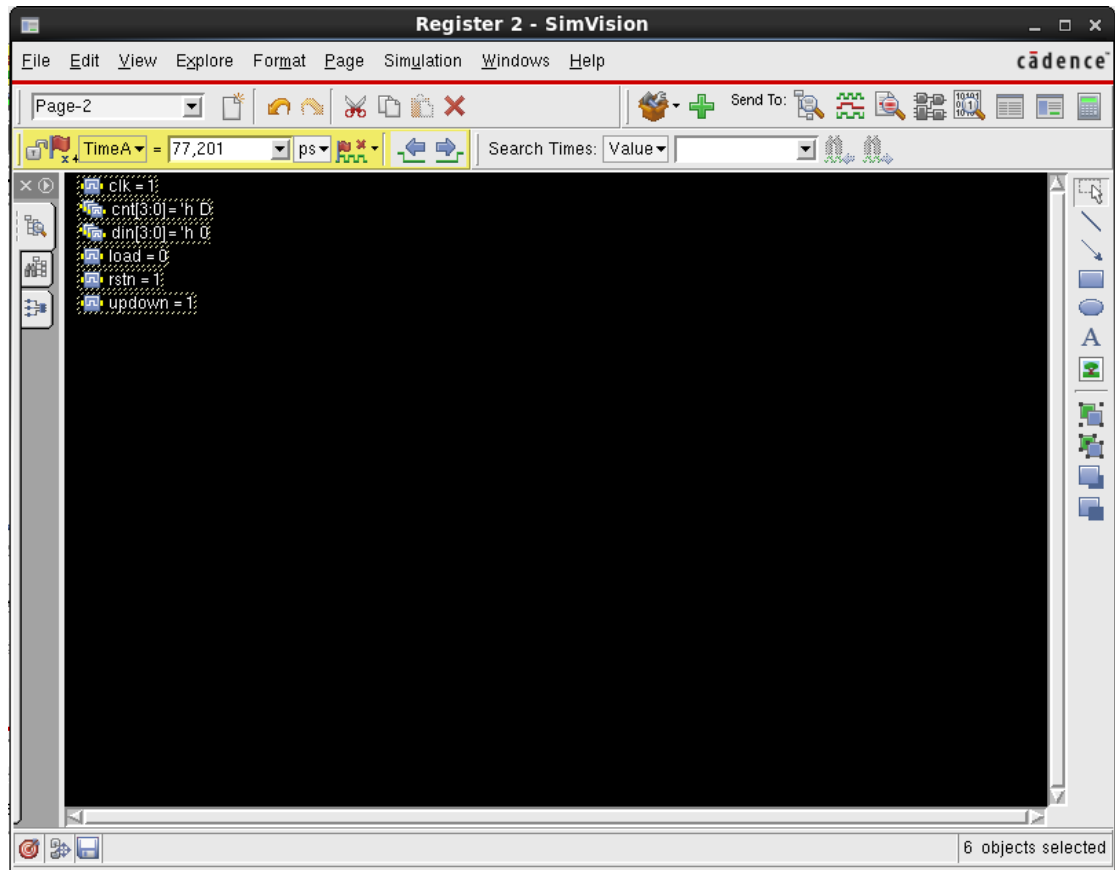


图12

(2)可以用寄存器窗口右侧按钮可以在窗口中自由地画图形化的图标与文本，用来管理目标布局，鼠标放在按钮上时，有关于按钮作用的快速提示。利用按钮工具画出简易的输入输出关系与状态图，如图13 所示，在TimeA=框中键入时间，界面将显示当前各信号的值。

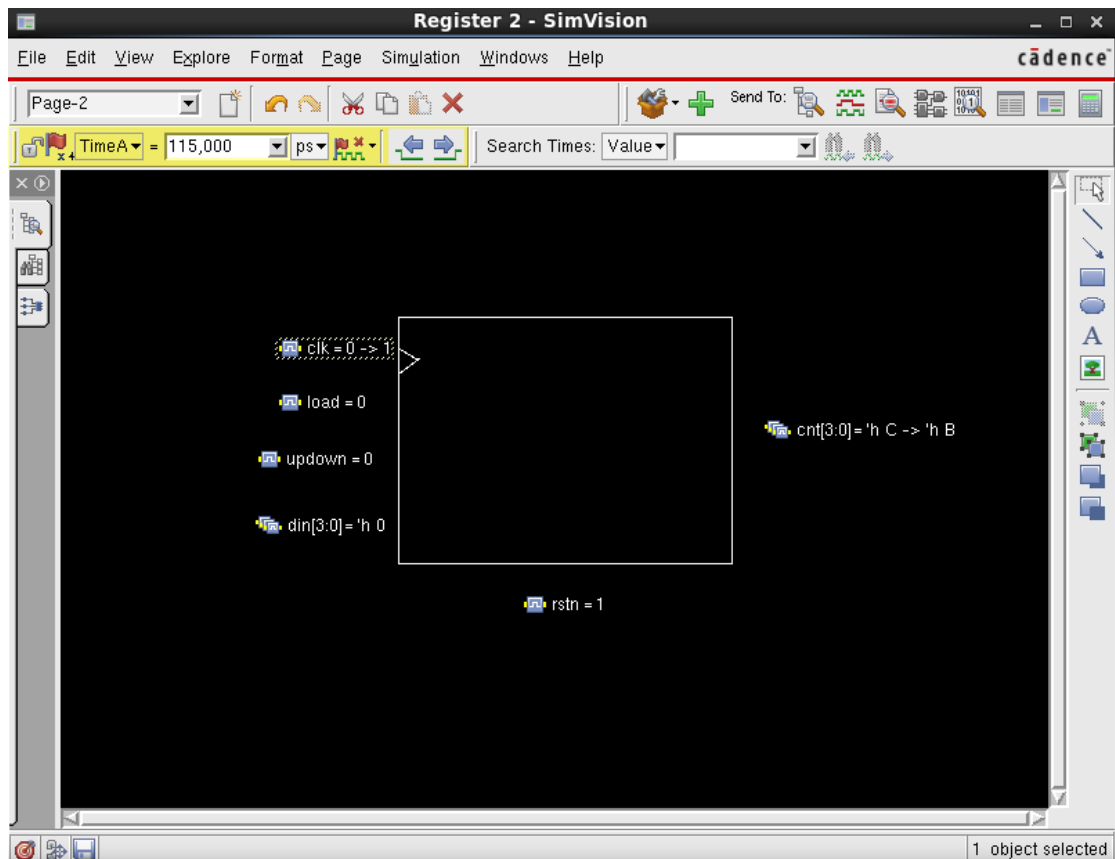


图13

(3)选中一个信号，如clk，点击图14 中所示按钮，可以快速定位此下一个跳变沿并给出当前各信号的值。

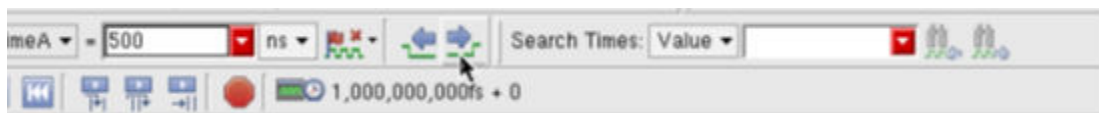


图14

(4)寄存器窗口在调试状态电路方面比较有优势，通过对时钟沿或者是状态信号跳变沿的判断，可以很直观的仿真出代码所述电路个状态之间的切换。

总结

关于软件的使用会有很多小技巧，同学们可以在学会基础操作的基础上自己摸索，一般的功能按钮在鼠标放上去的时候都会有相应的提示，对于信号的一些操作基本上右键都能解决。

附录：实验用代码

(1) demo-updown-cnt.v

(2) `timescale 1ns/1ps

```
module updown_cnt(clk,rstn,din,load,updown,cnt);
```

```

input clk,rstn;
input [3:0] din;
input load, updown;
output [3:0] cnt;
reg [3:0] cnt;

always@(posedge clk or negedge rstn)
begin
if(!rstn)  cnt <= 4'd0;
else if(load)  cnt <= din;
      else if(updown) cnt <= cnt + 1;
      else cnt <= cnt - 1;
end

endmodule

(3) demo-updown-cnt-tb.v
`timescale 1ns/1ps

module tb;

reg clk,rstn;
reg [3:0] din;
reg load, updown;
wire [3:0] cnt;

updown_cnt uut(clk,rstn,din,load,updown,cnt);

parameter half_cycle = 5;
initial
begin
clk = 0;
forever
clk = #half_cycle ~clk;
end

initial
begin
rstn = 1;
#(1.5*half_cycle) rstn = 0;
#(2*half_cycle) rstn = 1;
end

initial
begin

```

```
load = 0;  
#(4*half_cycle) load = 1;  
#(2*half_cycle) load = 0;  
end
```

```
initial  
begin  
din = 0;  
#(4*half_cycle) din = 4'd10;  
#(2*half_cycle) din = 0;  
end
```

```
initial  
begin  
updown = 0;  
#(8*half_cycle) updown = 1;  
#(10*half_cycle) updown = 0;  
end
```

```
initial  
#(50*half_cycle) $finish;
```

```
endmodule
```